

Programació recursiva metòdica (I)

Departament de Ciències de la Computació

Universitat Politècnica de Catalunya



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Politècnica Superior d'Enginyeria
de Vilanova i la Geltrú

Índex

- 1 Programes recursius
- 2 Correctesa de programes recursius
- 3 Exemples de disseny i justificació de programes recursius
- 4 Exercicis de disseny i justificació de programes recursius amb arbres binaris

Índex

- 1 Programes recursius
- 2 Correctesa de programes recursius
- 3 Exemples de disseny i justificació de programes recursius
- 4 Exercicis de disseny i justificació de programes recursius amb arbres binaris

Recursivitat I

- Una funció és recursiva quan es defineix en funció de si mateixa (pot cridar-se a si mateixa).

Recursivitat I

- Una funció és recursiva quan es defineix en funció de si mateixa (pot cridar-se a si mateixa).
- Alguns llenguatges de programació no permeten la recursivitat (pocs). El C++ sí.

Recursivitat I

- Una funció és recursiva quan es defineix en funció de si mateixa (pot cridar-se a si mateixa).
- Alguns llenguatges de programació no permeten la recursivitat (pocs). El C++ sí.
- Una funció recursiva cal que estigui dissenyada de forma especial perquè no s'executi indefinidament.

Recursivitat I

- Una funció és recursiva quan es defineix en funció de si mateixa (pot cridar-se a si mateixa).
- Alguns llenguatges de programació no permeten la recursivitat (pocs). El C++ sí.
- Una funció recursiva cal que estigui dissenyada de forma especial perquè no s'executi indefinidament.
- A cada crida recursiva es consumeixen nous recursos. Es creen noves variables locals diferents de les anteriors (les variables i paràmetres anteriors es guarden, usant una pila, per recuperar-los al tornar).

Recursivitat I

- Una funció és recursiva quan es defineix en funció de si mateixa (pot cridar-se a si mateixa).
- Alguns llenguatges de programació no permeten la recursivitat (pocs). El C++ sí.
- Una funció recursiva cal que estigui dissenyada de forma especial perquè no s'executi indefinidament.
- A cada crida recursiva es consumeixen nous recursos. Es creen noves variables locals diferents de les anteriors (les variables i paràmetres anteriors es guarden, usant una pila, per recuperar-los al tornar). ✗

Recursivitat II

- Es pot demostrar que tota funció recursiva també pot escriure's de forma iterativa i viceversa. . . Per què la recursivitat?

Recursivitat II

- Es pot demostrar que tota funció recursiva també pot escriure's de forma iterativa i viceversa. . . Per què la recursivitat?
 - Disseny més abstracte i compacte.

Recursivitat II

- Es pot demostrar que tota funció recursiva també pot escriure's de forma iterativa i viceversa. . . Per què la recursivitat?
 - Disseny més abstracte i compacte.
 - De vegades, disseny molt més simple i natural que la iteració.

Recursivitat II

- Es pot demostrar que tota funció recursiva també pot escriure's de forma iterativa i viceversa. . . Per què la recursivitat?
 - Disseny més abstracte i compacte.
 - De vegades, disseny molt més simple i natural que la iteració.
- Quan dissenyem una funció recursiva hem de raonar “en recursiu”, no intentant seguir la seva execució.

Estructura d'un programa recursiu

- Les funcions recursives sempre tenen la mateixa estructura:

Estructura d'un programa recursiu

- Les funcions recursives sempre tenen la mateixa estructura:
 - Un o més casos base, que són casos trivials de solució immediata.

Estructura d'un programa recursiu

- Les funcions recursives sempre tenen la mateixa estructura:
 - Un o més casos base, que són casos trivials de solució immediata.
 - Un o més casos recursius, que són casos on la solució del problema s'expressa en funció de les solucions del mateix problema aplicat a casos “més petits”.

Estructura d'un programa recursiu

- Les funcions recursives sempre tenen la mateixa estructura:
 - Un o més casos base, que són casos trivials de solució immediata.
 - Un o més casos recursius, que són casos on la solució del problema s'expressa en funció de les solucions del mateix problema aplicat a casos "més petits".
- Per assegurar que una funció recursiva acaba, cal garantir que el paràmetre, o combinació de paràmetres, es va fent més petit a cada crida recursiva.

Esquema bàsic d'un programa recursiu

```

Tipus2 f(Tipus1 x)
/* Pre: Q(x) */
/* Post: R(x, s) */
{
    Tipus2 r, s;

    if (c(x)) s = d(x);
    else {
        r = f(g(x));
        s = h(x, r);
    }
    return s;
}

```

Esquema bàsic d'un programa recursiu

```

Tipus2 f(Tipus1 x)
/* Pre: Q(x) */
/* Post: R(x, s) */
{
    Tipus2 r, s;

    if (c(x)) s = d(x);
    else {
        r = f(g(x));
        s = h(x, r);
    }
    return s;
}

```

- $d(x)$: cas base o directe. Podria haver-n'hi més d'un.

Esquema bàsic d'un programa recursiu

```

Tipus2 f(Tipus1 x)
/* Pre: Q(x) */
/* Post: R(x, s) */
{
    Tipus2 r, s;

    if (c(x)) s = d(x);
    else {
        r = f(g(x));
        s = h(x, r);
    }
    return s;
}

```

- $d(x)$: cas base o directe. Podria haver-n'hi més d'un.
- $f(g(x))$: cas recursiu. Crida a la pròpia funció f però amb els paràmetres "més petits" (després d'aplicar-los una funció g). Podria haver-n'hi més d'un (amb g diferents).

Esquema bàsic d'un programa recursiu

```

Tipus2 f(Tipus1 x)
/* Pre: Q(x) */
/* Post: R(x, s) */
{
    Tipus2 r, s;

    if (c(x)) s = d(x);
    else {
        r = f(g(x));
        s = h(x, r);
    }
    return s;
}

```

- $d(x)$: cas base o directe. Podria haver-n'hi més d'un.
- $f(g(x))$: cas recursiu. Crida a la pròpia funció f però amb els paràmetres "més petits" (després d'aplicar-los una funció g). Podria haver-n'hi més d'un (amb g diferents).
- $h(x, r)$: instruccions addicionals després de la crida recursiva. Podria ser $\emptyset$.

Exemple: factorial d'un nombre enter

$$n! = \begin{cases} 1, & n = 0 \\ (n - 1)! \cdot n, & n > 0 \end{cases}$$

Exemple: factorial d'un nombre enter

$$n! = \begin{cases} 1, & n = 0 \\ (n - 1)! \cdot n, & n > 0 \end{cases}$$

- Cas directe: quan puguem donar el resultat directament, sense haver de fer cap crida recursiva.

Exemple: factorial d'un nombre enter

$$n! = \begin{cases} 1, & n = 0 \\ (n - 1)! \cdot n, & n > 0 \end{cases}$$

- Cas directe: quan puguem donar el resultat directament, sense haver de fer cap crida recursiva. **En el cas del factorial, 0!.**

Exemple: factorial d'un nombre enter

$$n! = \begin{cases} 1, & n = 0 \\ (n - 1)! \cdot n, & n > 0 \end{cases}$$

- Cas directe: quan puguem donar el resultat directament, sense haver de fer cap crida recursiva. **En el cas del factorial, 0!**.
- Cas recursiu: quan sigui necessari fer una crida recursiva per obtenir el resultat de la funció però amb unes dades “més petites”.

Exemple: factorial d'un nombre enter

$$n! = \begin{cases} 1, & n = 0 \\ (n - 1)! \cdot n, & n > 0 \end{cases}$$

- Cas directe: quan puguem donar el resultat directament, sense haver de fer cap crida recursiva. **En el cas del factorial, 0!.**
- Cas recursiu: quan sigui necessari fer una crida recursiva per obtenir el resultat de la funció però amb unes dades “més petites”.
Per calcular $n!$ ens cal tenir calculat $(n - 1)!$.

Exemple: factorial d'un nombre enter

$$n! = \begin{cases} 1, & n = 0 \\ (n - 1)! \cdot n, & n > 0 \end{cases}$$

- Cas directe: quan puguem donar el resultat directament, sense haver de fer cap crida recursiva. **En el cas del factorial, 0!.**
- Cas recursiu: quan sigui necessari fer una crida recursiva per obtenir el resultat de la funció però amb unes dades “més petites”. **Per calcular $n!$ ens cal tenir calculat $(n - 1)!$.**
- Instruccions addicionals: al tornar de la crida recursiva no tenim el resultat final, cal fer alguna operació addicional per obtenir-lo.

Exemple: factorial d'un nombre enter

$$n! = \begin{cases} 1, & n = 0 \\ (n - 1)! \cdot n, & n > 0 \end{cases}$$

- Cas directe: quan puguem donar el resultat directament, sense haver de fer cap crida recursiva. **En el cas del factorial, 0!.**
- Cas recursiu: quan sigui necessari fer una crida recursiva per obtenir el resultat de la funció però amb unes dades “més petites”. **Per calcular $n!$ ens cal tenir calculat $(n - 1)!$.**
- Instruccions addicionals: al tornar de la crida recursiva no tenim el resultat final, cal fer alguna operació addicional per obtenir-lo. **Per calcular $n!$ ens cal multiplicar $(n - 1)!$ per n .**

Exemple: càlcul del factorial

```
int factorial(int n)
/* Pre: n >= 0 */
/* Post: el resultat
retornat és n! */
{
    int f, faux;
    if (n == 0) f=1;
    else {
        faux = factorial(n - 1);
        f = n * faux;
    }
    return f;
}
```

```
c(n) = n == 0
d(n) = 1
g(n) = n-1
h(n, faux) = n * faux;
```

Índex

- 1 Programes recursius
- 2 **Correctesa de programes recursius**
- 3 Exemples de disseny i justificació de programes recursius
- 4 Exercicis de disseny i justificació de programes recursius amb arbres binaris

Principi d'inducció: definició

- Per demostrar que un programa és correcte amb un nombre finit de proves cal un mecanisme d'inducció.

Principi d'inducció: definició

- Per demostrar que un programa és correcte amb un nombre finit de proves cal un mecanisme d'inducció.
- El Principi d'Inducció (matemàtiques) serveix per demostrar propietats que valen per tot nombre natural.

Principi d'inducció: definició

- Per demostrar que un programa és correcte amb un nombre finit de proves cal un mecanisme d'inducció.
- El Principi d'Inducció (matemàtiques) serveix per demostrar propietats que valen per tot nombre natural.

Principi d'inducció (en els naturals)

P és una propietat que es pot satisfer o no per cada nombre natural.

- 1 Si el número 0 compleix la propietat P , i
- 2 quan un nombre natural n compleix la propietat P , aleshores també la compleix el nombre següent $n + 1$

aleshores tots els nombres naturals compleixen la propietat P .

Demostracions per inducció

- Per demostrar que tot natural compleix una propietat usant aquest principi, caldrà provar: (1) que 0 la compleix i (2) que si n la compleix, aleshores $n + 1$ també la compleix.

Demostracions per inducció

- Per demostrar que tot natural compleix una propietat usant aquest principi, caldrà provar: (1) que 0 la compleix i (2) que si n la compleix, aleshores $n + 1$ també la compleix.
- *Exemple:* demostreu que per tot nombre natural n , $n^2 + 5n + 6$ és un nombre parell.

Demostracions per inducció

- Per demostrar que tot natural compleix una propietat usant aquest principi, caldrà provar: (1) que 0 la compleix i (2) que si n la compleix, aleshores $n + 1$ també la compleix.
- *Exemple:* demostreu que per tot nombre natural n , $n^2 + 5n + 6$ és un nombre parell.

Demostració

- Per $n = 0$: $0^2 + 5 \cdot 0 + 6$ és parell.

Demostracions per inducció

- Per demostrar que tot natural compleix una propietat usant aquest principi, caldrà provar: (1) que 0 la compleix i (2) que si n la compleix, aleshores $n + 1$ també la compleix.
- *Exemple:* demostreu que per tot nombre natural n , $n^2 + 5n + 6$ és un nombre parell.

Demostració

- Per $n = 0$: $0^2 + 5 \cdot 0 + 6$ és parell. ✓

Demostracions per inducció

- Per demostrar que tot natural compleix una propietat usant aquest principi, caldrà provar: (1) que 0 la compleix i (2) que si n la compleix, aleshores $n + 1$ també la compleix.
- *Exemple:* demostreu que per tot nombre natural n , $n^2 + 5n + 6$ és un nombre parell.

Demostració

- Per $n = 0$: $0^2 + 5 \cdot 0 + 6$ és parell. ✓
- $n^2 + 5n + 6$ és parell $\Rightarrow (n + 1)^2 + 5(n + 1) + 6$ és parell?

Demostracions per inducció

- Per demostrar que tot natural compleix una propietat usant aquest principi, caldrà provar: (1) que 0 la compleix i (2) que si n la compleix, aleshores $n + 1$ també la compleix.
- *Exemple:* demostreu que per tot nombre natural n , $n^2 + 5n + 6$ és un nombre parell.

Demostració

- Per $n = 0$: $0^2 + 5 \cdot 0 + 6$ és parell. ✓
- $n^2 + 5n + 6$ és parell $\Rightarrow (n + 1)^2 + 5(n + 1) + 6$ és parell? $(n + 1)^2 + 5(n + 1) + 6 = n^2 + 5n + 6 + (2n + 6)$, la suma de parells és parell

Demostracions per inducció

- Per demostrar que tot natural compleix una propietat usant aquest principi, caldrà provar: (1) que 0 la compleix i (2) que si n la compleix, aleshores $n + 1$ també la compleix.
- *Exemple:* demostreu que per tot nombre natural n , $n^2 + 5n + 6$ és un nombre parell.

Demostració

- Per $n = 0$: $0^2 + 5 \cdot 0 + 6$ és parell. ✓
- $n^2 + 5n + 6$ és parell $\Rightarrow (n + 1)^2 + 5(n + 1) + 6$ és parell? $(n + 1)^2 + 5(n + 1) + 6 = n^2 + 5n + 6 + (2n + 6)$, la suma de parells és parell ✓

Justificació de la correctesa de programes recursius

- Com apliquem el principi d'inducció per demostrar que un programa recursiu és correcte?

Justificació de la correctesa de programes recursius

- Com apliquem el principi d'inducció per demostrar que un programa recursiu és correcte?
 - 1 Si a l'inici de l'execució les variables satisfan la precondition, en acabar l'execució satisfan la postcondició (per inducció).

Justificació de la correctesa de programes recursius

- Com apliquem el principi d'inducció per demostrar que un programa recursiu és correcte?
 - 1 Si a l'inici de l'execució les variables satisfan la precondició, en acabar l'execució satisfan la postcondició (per inducció).
 - 2 Demostrar que l'execució acabarà (funció de fita).

Demostrar la correctesa dels programes recursius

Recordatori esquema general funció recursiva

```

Tipus2 f(Tipus1 x)
/* Pre: Q(x) */
/* Post: R(x, s) */
{
    Tipus2 r, s;

    if (c(x)) s = d(x);
    else {
        r = f(g(x));
        s = h(x, r);
    }
    return s;
}

```

Demostrar la correctesa dels programes recursius

Pas 1 Correctesa cas directe: $Q(x) \text{ and } c(x) \implies R(x, d(x))$

Demostrar la correctesa dels programes recursius

Pas 1 Correctesa cas directe: $Q(x) \text{ and } c(x) \implies R(x, d(x))$

Pas 2 Hipòtesi d'inducció: $Q(g(x)) \implies R(g(x), r)$

Demostrar la correctesa dels programes recursius

Pas 1 Correctesa cas directe: $Q(x) \text{ and } c(x) \implies R(x, d(x))$

Pas 2 Hipòtesi d'inducció: $Q(g(x)) \implies R(g(x), r)$

Pas 3 Correctesa del cas recursiu (o casos recursius):

Demostrar la correctesa dels programes recursius

Pas 1 Correctesa cas directe: $Q(x) \text{ and } c(x) \implies R(x, d(x))$

Pas 2 Hipòtesi d'inducció: $Q(g(x)) \implies R(g(x), r)$

Pas 3 Correctesa del cas recursiu (o casos recursius):

Pas 3.1 Correctesa crida recursiva:

$Q(x) \text{ and not } c(x) \implies Q(g(x))$

Demostrar la correctesa dels programes recursius

Pas 1 Correctesa cas directe: $Q(x) \text{ and } c(x) \implies R(x, d(x))$

Pas 2 Hipòtesi d'inducció: $Q(g(x)) \implies R(g(x), r)$

Pas 3 Correctesa del cas recursiu (o casos recursius):

Pas 3.1 Correctesa crida recursiva:

$$Q(x) \text{ and not } c(x) \implies Q(g(x))$$

Pas 3.2 Pas d'inducció:

$$Q(x) \text{ and not } c(x) \text{ and } R(g(x), r) \implies R(x, h(x, r))$$

Demostrar la correctesa dels programes recursius

Pas 1 Correctesa cas directe: $Q(x) \text{ and } c(x) \implies R(x, d(x))$

Pas 2 Hipòtesi d'inducció: $Q(g(x)) \implies R(g(x), r)$

Pas 3 Correctesa del cas recursiu (o casos recursius):

Pas 3.1 Correctesa crida recursiva:

$$Q(x) \text{ and not } c(x) \implies Q(g(x))$$

Pas 3.2 Pas d'inducció:

$$Q(x) \text{ and not } c(x) \text{ and } R(g(x), r) \implies R(x, h(x, r))$$

Pas 4 Acabament de la recursivitat:

Demostrar la correctesa dels programes recursius

Pas 1 Correctesa cas directe: $Q(x) \text{ and } c(x) \implies R(x, d(x))$

Pas 2 Hipòtesi d'inducció: $Q(g(x)) \implies R(g(x), r)$

Pas 3 Correctesa del cas recursiu (o casos recursius):

Pas 3.1 Correctesa crida recursiva:

$$Q(x) \text{ and not } c(x) \implies Q(g(x))$$

Pas 3.2 Pas d'inducció:

$$Q(x) \text{ and not } c(x) \text{ and } R(g(x), r) \implies R(x, h(x, r))$$

Pas 4 Acabament de la recursivitat:

Pas 4.1 Existeix una funció de fita $t(x)$ t.q. $Q(x) \implies t(x) \in \mathbb{N}$

Demostrar la correctesa dels programes recursius

Pas 1 Correctesa cas directe: $Q(x)$ and $c(x) \implies R(x, d(x))$

Pas 2 Hipòtesi d'inducció: $Q(g(x)) \implies R(g(x), r)$

Pas 3 Correctesa del cas recursiu (o casos recursius):

Pas 3.1 Correctesa crida recursiva:

$$Q(x) \text{ and not } c(x) \implies Q(g(x))$$

Pas 3.2 Pas d'inducció:

$$Q(x) \text{ and not } c(x) \text{ and } R(g(x), r) \implies R(x, h(x, r))$$

Pas 4 Acabament de la recursivitat:

Pas 4.1 Existeix una funció de fita $t(x)$ t.q. $Q(x) \implies t(x) \in \mathbb{N}$

Pas 4.2 i decreix a cada crida recursiva:

$$Q(x) \text{ and not } c(x) \implies t(g(x)) < t(x)$$

Índex

- 1 Programes recursius
- 2 Correctesa de programes recursius
- 3 Exemples de disseny i justificació de programes recursius
- 4 Exercicis de disseny i justificació de programes recursius amb arbres binaris

Exemple 1: piles iguals

- Donades dues piles, comproveu si són iguals, és a dir, si tenen els mateixos elements en el mateix ordre.

Exemple 1: piles iguals

- Donades dues piles, comproveu si són iguals, és a dir, si tenen els mateixos elements en el mateix ordre.
- Especificació:

```
bool piles_iguals(stack<int> &p1, stack<int> &p2);
/* Pre:  p1 = P1 i p2 = P2 */
/* Post: El resultat ens indica si les dues piles
        P1 i P2 són iguals */
```

Solució: piles iguals

```
bool piles_iguals(stack<int> &p1, stack<int> &p2 )
/* Pre: p1 = P1 i p2 = P2 */
/* Post: El resultat ens indica si les dues piles
P1 i P2 són iguals */
{
    bool iguals;
    if (p1.empty() or p2.empty()) iguals = p1.empty() and p2.empty();
    else if (p1.top() != p2.top()) iguals = false;
    else {
        p1.pop();
        p2.pop();
        iguals = piles_iguals(p1, p2);
        /* HI: iguals = "P1 sense l'últim element afegit
i P2 sense l'últim element afegit són dues
piles iguals" */
    }
    return iguals;
}
```

Justificació informal: piles iguals

● Casos senzills:

- Si alguna de les dues piles està buida, llavors les piles són iguals si les dues estan buides. Assignem a `iguals` l'expressió `p1.empty() and p2.empty()`.
- Si les dues piles no estan buides, podem consultar els seus cims i, si són diferents, llavors les piles no poden ser iguals. Posem `iguals` a fals.

Justificació informal: piles iguals

- Casos senzills:
 - Si alguna de les dues piles està buida, llavors les piles són iguals si les dues estan buides. Assignem a `iguals` l'expressió `p1.empty() and p2.empty()`.
 - Si les dues piles no estan buides, podem consultar els seus cims i, si són diferents, llavors les piles no poden ser iguals. Posem `iguals` a fals.
- Cas recursiu: Si les piles no estan buides i els seus cims són iguals, llavors les piles són iguals si coincideixen en la resta d'elements. Això ho podem obtenir, per **hipòtesi d'inducció**, cridant recursivament la funció amb les piles sense l'element del cim. Assignem a `iguals` el resultat de la crida recursiva.

Justificació informal: piles iguals

- Casos senzills:
 - Si alguna de les dues piles està buida, llavors les piles són iguals si les dues estan buides. Assignem a `iguals` l'expressió `p1.empty() and p2.empty()`.
 - Si les dues piles no estan buides, podem consultar els seus cims i, si són diferents, llavors les piles no poden ser iguals. Posem `iguals` a fals.
- Cas recursiu: Si les piles no estan buides i els seus cims són iguals, llavors les piles són iguals si coincideixen en la resta d'elements. Això ho podem obtenir, per **hipòtesi d'inducció**, cridant recursivament la funció amb les piles sense l'element del cim. Assignem a `iguals` el resultat de la crida recursiva.
- Acabament: A cada crida recursiva fem més petita la primera pila (i la segona).

Exercici: cerca en una cua de punts

```
bool cerca_rec_cua_punt(queue<Punt> &c, float x)
/* Pre: c = C */
/* Post: El resultat indica si la cua C conté algun
punt amb la primera coordenada igual a x */
{
    bool trobat;
    if (...) trobat = ...;
    else {
        ...
        trobat = ...;
    }
    return trobat;
}
```

Exemple 2: mida d'un arbre binari (I)

```
int mida(const arbreBin<int> &a)
/* Pre: a = A */
/* Post: El resultat és el nombre de nodes de l'arbre A */
{
    int nnodes;
    if (...) nnodes = ...;
    else {
        ...
        nnodes = ...;
    }
    return nnodes;
}
```

Exemple 2: mida d'un arbre binari (II)

```

int mida(const arbreBin<int> &a)
/* Pre: a = A */
/* Post: El resultat és el nombre de nodes de l'arbre A */
{
    int nnodes;
    if (a.es_buit()) nnodes = 0;
    else {
        arbreBin<int> a1 = a.fe();
        arbreBin<int> a2 = a.fd();
        int y = mida(a1);
        int z = mida(a2);
        /* H1: y = "nombre de nodes del fill esquerre d'A" i
           z = "nombre de nodes del fill dret d'A" */
        nnodes = y + z + 1;
    }
    return nnodes;
}

```

Justificació informal: mida d'un arbre binari

- Cas senzill:
 - Si l'arbre està buit, té zero elements. Assignem a `nnodes` aquest valor.

Justificació informal: mida d'un arbre binari

- Cas senzill:
 - Si l'arbre està buit, té zero elements. Assignem a `nnodes` aquest valor.
- Cas recursiu: Si l'arbre no està buit, llavors té fills que podem obtenir amb les operacions `fe()` i `fd()`. La mida de l'arbre serà la suma de la mida dels seus subarbres més 1 (l'arrel).
Les mides dels subarbres s'obtenen per hipòtesi d'inducció amb les crides recursives.

Justificació informal: mida d'un arbre binari

- Cas senzill:
 - Si l'arbre està buit, té zero elements. Assignem a `nnodes` aquest valor.
- Cas recursiu: Si l'arbre no està buit, llavors té fills que podem obtenir amb les operacions `fe()` i `fd()`. La mida de l'arbre serà la suma de la mida dels seus subarbres més 1 (l'arrel).
Les mides dels subarbres s'obtenen per hipòtesi d'inducció amb les crides recursives.
- Acabament: A cada crida recursiva fem més petit l'arbre.

Índex

- 1 Programes recursius
- 2 Correctesa de programes recursius
- 3 Exemples de disseny i justificació de programes recursius
- 4 Exercicis de disseny i justificació de programes recursius amb arbres binaris**

Exercicis de disseny i justificació de programes recursius amb arbres binaris

Nota: Per aquests exercicis, suposarem que els arbres binaris estan instanciats amb enters (`arbreBin<int>`).

- Exercici 1: Calcular el nombre d'aparicions d'un element en un arbre binari.
- Exercici 2: Cercar un valor donat en un arbre binari.
- Exercici 3: Sumar un valor k a tots els nodes d'un arbre binari.
- Exercici 4: Donat un arbre binari, substituir tota aparició del valor x pel valor y .